

halfedge：连通分量分析模块设计文档

src/connectivity.rs

halfedge 库

2026-07-04

目录

1	模块定位	1
1.1	API 总览	2
2	连通定义	2
2.1	面连通	2
2.2	顶点连通	2
3	算法：广度优先搜索（BFS）	3
3.1	component_of_face 流程	3
3.2	connected_components 与 component_count	3
3.3	vertex_connected_components	3
4	网格拆分与合并	3
4.1	extract_faces：子网格提取	3
4.2	split_into_components：多分量拆分	4
4.3	merge_meshes：网格合并	4
4.4	拆分/合并流程	4
5	复杂度	4
6	退化守卫	5
7	测试覆盖	5
8	设计权衡	6

1 模块定位

connectivity.rs 在 storage.rs 与 traversal.rs 之上提供连通分量检测与网格拆分/合并能力，回答以下两类问题：

- **面连通分量**：哪些面通过共享边互相可达？用于分离网格中互不相连的「片」（如 OBJ 文件中的多个独立物体被一次性载入）。
- **顶点连通分量**：哪些顶点通过半边邻接互相可达？用于检测孤立顶点、顶点级连通性分析。
- **网格拆分/合并**：将多分量网格拆成独立 `MeshStorage`，或将多个网格合并为一个。

1.1 API 总览

类别	API	语义
面分量	<code>connected_components(mesh)</code>	返回 <code>Vec<Vec<FaceId>></code> ，所有面分量
	<code>component_of_face(mesh, seed)</code>	返回包含 <code>seed</code> 的面分量， <code>Option</code>
	<code>component_count(mesh)</code>	仅返回面分量数量（不分配分量列表）
顶点分量	<code>vertex_connected_components(mesh)</code>	返回 <code>Vec<Vec<VertexId>></code> ，所有顶点分量
分量提取	<code>extract_component(mesh, seed)</code>	提取 <code>seed</code> 所在分量为独立 <code>MeshStorage</code>
	<code>extract_faces(mesh, &[f])</code>	提取指定面集合为独立 <code>MeshStorage</code>
拆分	<code>split_into_components(mesh)</code>	按面连通分量拆分为多个独立 <code>MeshStorage</code>
合并	<code>merge_meshes(a, b)</code>	将两个网格合并为一个（拓扑独立，仅共享存储）

2 连通定义

2.1 面连通

两个面 f_1, f_2 **直接连通** 当且仅当它们共享一条边（即共享两个顶点）：

$$f_1 \sim f_2 \iff |V(f_1) \cap V(f_2)| \geq 2.$$

面连通分量 是 $\sim$ 的传递闭包生成的等价类。注意：仅共享一个顶点不算连通（两个面必须共享一条边才被认为是同一分量）。

2.2 顶点连通

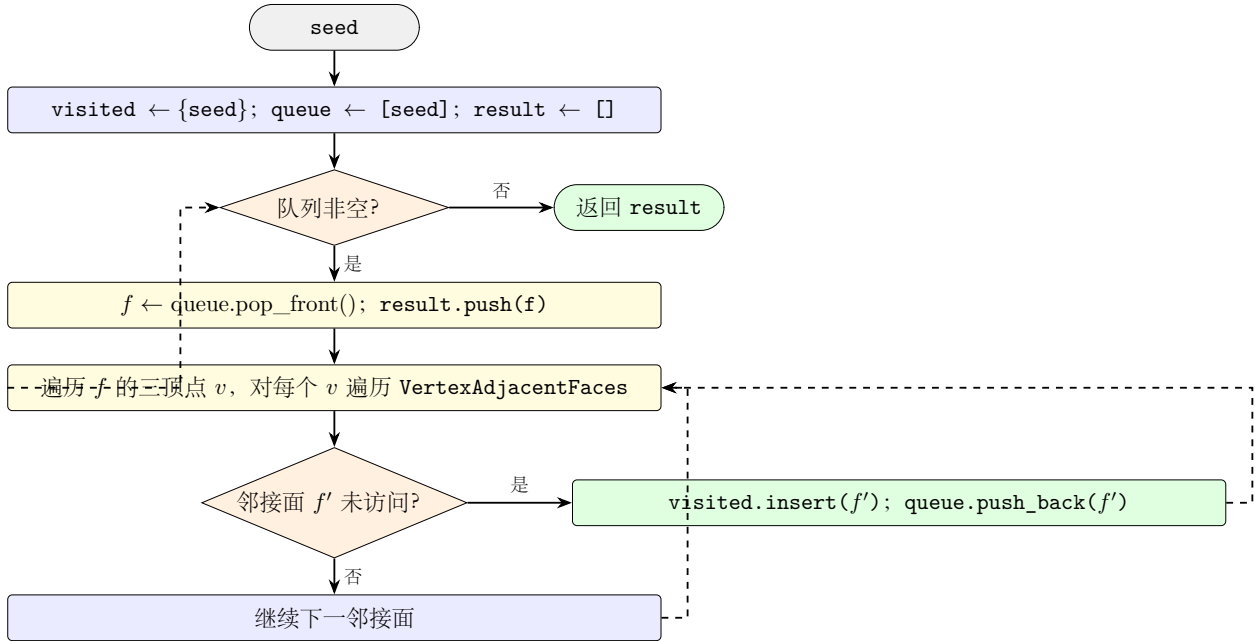
两个顶点 u, v **直接连通** 当且仅当它们是一条半边的两 endpoint：

$$u \sim v \iff \exists h \in H, \text{src}(h) = u \wedge \text{dst}(h) = v.$$

顶点连通分量同样由 $\sim$ 的传递闭包生成。

3 算法：广度优先搜索（BFS）

3.1 component_of_face 流程



3.2 connected_components 与 component_count

`connected_components` 对每个未访问的 `seed` 调用 `component_of_face`, 合并所有分量。`component_count` 是其简化版本, 仅返回 `.len()`。

3.3 vertex_connected_components

顶点版本使用 `VertexAdjacentVerts` 迭代器(基于顶点 outgoing 半边环)替代 `VertexAdjacentFaces`, BFS 结构完全一致。

4 网格拆分与合并

4.1 extract_faces: 子网格提取

`extract_faces(mesh, faces)` 将指定面集合提取为独立 `MeshStorage`, 保留内部拓扑; 与外部相邻的边在新网格中成为**边界边**。

算法

1. 收集 `faces` 中所有半边 (`he_set`) 与顶点 (`vert_set`);
2. 在新 `MeshStorage` 中按 `vert_set` 创建顶点, 建立 `v_map`;
3. 按 `he_set` 创建半边 (`vertex` 字段经 `v_map` 映射), 建立 `he_map`;
4. 复制半边 `twin/next/prev` 字段, 仅当引用的半边也在 `he_map` 中时映射, 否则置为 `None` (外部 `twin` 切断 $\rightarrow$ 边界边);

5. 创建面，映射 `halfedge` 字段与半边 `face` 字段；
6. 修复顶点 `outgoing` 半边入口（若原 `outgoing` 不在分量中，置 `None`）。

`extract_component(mesh, seed)` 是 `component_of_face + extract_faces` 的简写。

4.2 split_into_components: 多分量拆分

$$\text{split_into_components}(\text{mesh}) = [\text{extract_faces}(\text{mesh}, c_i)]_{i=1}^k$$

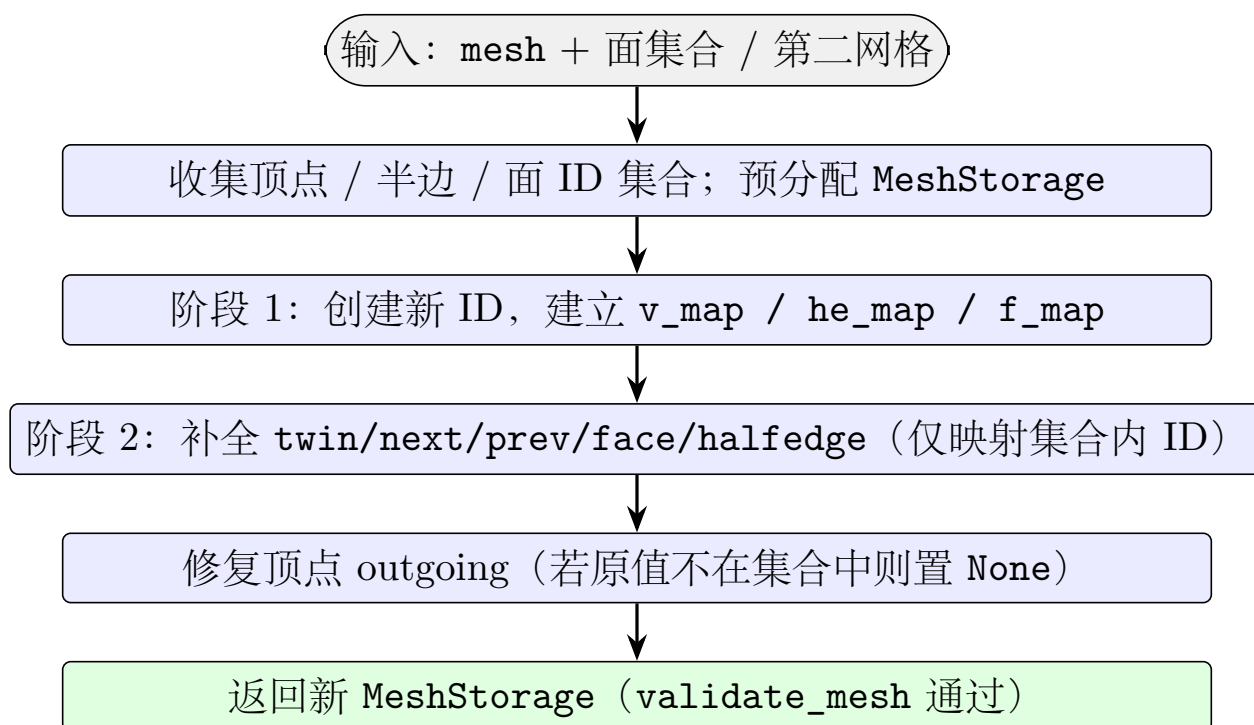
其中 $c_1, \dots, c_k$ 是 `connected_components` 的返回值。每个分量通过 `validate_mesh` 校验。

4.3 merge_meshes: 网格合并

`merge_meshes(a, b)` 创建新 `MeshStorage`，依次将 `a` 与 `b` 的所有元素复制进去，建立各自的 ID 映射。两网格拓扑**保持独立**：合并后 `a` 与 `b` 之间不建立新的 `twin` 连接，仅共享同一个 `MeshStorage` 容器。常用于场景组装、批量导出。

内部辅助 `copy_mesh_into(src, dst, v_map, he_map, f_map)` 是被 `merge_meshes` 与未来可能的 `transform` 类操作复用的核心函数，按「先创建 ID 映射，再补全字段」两阶段完成。

4.4 拆分/合并流程



5 复杂度

API	时间复杂度	空间复杂度
<code>connected_components</code>	$O(V + F)$	$O(F)$ (visited + 队列 + 结果)
<code>component_of_face</code>	$O(V_f + F_c)$	$O(F_c)$ (F_c = 分量大小)
<code>component_count</code>	$O(V + F)$	$O(F)$
<code>vertex_connected_components</code>	$O(V + E)$	$O(V)$
<code>extract_faces</code>	$O(F_c)$	$O(V_c + E_c + F_c)$ (新网格 + 三张映射)
<code>extract_component</code>	$O(V_f + F_c)$	同 <code>extract_faces</code>
<code>split_into_components</code>	$O(V + F)$	$O(V + E + F)$ (所有分量独立存储)
<code>merge_meshes</code>	$O(V_a + E_a + F_a + V_b + E_b + F_b)$	同左 (合并后总规模)

其中 V_f 表示 BFS 访问到的所有顶点数。每个面访问 3 顶点，`VertexAdjacentFaces` 内部为 $O(1)$ 查询，故整体线性。

6 退化守卫

- `component_of_face` 对无效 `seed` 返回 `None`，不 panic；
- 空网格的 `connected_components` 返回空 `Vec`；
- `HashSet<FaceId>` 防止重复访问，BFS 必终止。

7 测试覆盖

测试名	验证点
连通分量	
<code>single_triangle_one_component</code>	icosphere(0) 20 面全部连通，分量数 = 1
<code>two_disconnected_triangles</code>	两个不相交三角形，分量数 = 2
<code>component_of_face_invalid</code>	无效 <code>FaceId</code> 返回 <code>None</code>
<code>empty_mesh_no_components</code>	空网格面/顶点分量均为 0
拆分 / 提取 / 合并	
<code>split_two_disconnected_triangles</code>	双三角形拆分为 2 个独立网格，各 1 面 3 顶点
<code>extract_component_basic</code>	提取单分量后 <code>validate_mesh</code> 通过
<code>merge_two_icospheres</code>	两 icosphere 合并: $F = 40$, $V = 24$
<code>merge_preserves_boundaries</code>	合并后两三角形各自 3 条边界边保留

全模块共 8 个单元测试，覆盖连通检测、拆分、提取、合并四类场景；项目整体 `cargo test` 共 371 个用例。

8 设计权衡

- **BFS vs DFS:** BFS 在 `VecDeque` 上实现，避免 DFS 的递归栈溢出风险，对超大连通分量更稳定；
- **`component_count` 仍调用 `connected_components`:** 实现上未单独优化，因为复杂度已线性，且 `.len()` 调用零额外开销。如未来需要频繁调用，可改为不收集分量的版本；
- **面连通定义:** 本项目采用「共享一条边」的严格定义(两个顶点),与 `traversal::VertexAdjacentFaces` 的语义一致；若需「共享一个顶点」的弱定义，需要单独实现。